WEEK 05

JavaScript:

Advanced Concepts

Web Applications Development · Front-End Track

TODAY'S AGENDA

- §1 Scope & Closures
- §2 Higher-Order Functions
- §3 The 'this' Keyword & Arrows
- §4 Promises & Async/Await
- §5 The Event Loop
- §6 ES6+ Power Features
- §7 Error Handling & Defensive Coding

© Cyber.SoHo

Course Roadmap

Where we are in the semester

Week	Topic	Status
Week 03	HTML & CSS Fundamentals	
Week 04	JavaScript Basics & the DOM	
Week 05 ★	JavaScript: Advanced Concepts	← YOU ARE HERE
Week 06	Introduction to React	
Week 07	APIs & Data Fetching	

Every React/Vue/Angular concept you will encounter from Week 06 onwards is directly built on the patterns taught today.

Why This Week Matters

The gap between using JavaScript and truly understanding it

- Google Maps — Closures & async data fetching power every map tile request
- Spotify Web Player — Event Loop + Promises manage audio buffers + API calls
- Instagram Web — HOFs (map/filter) render your feed; React hooks use closures
- Your university's Moodle portal — async/await fetches grades + assignment data
- Every React, Vue, and Angular component ever written — HOFs, closures, async

After today you will not just USE JavaScript — you will understand WHY it works.

That mental model is what separates junior from mid-level engineers.

SECTION 1 · 8 MIN

Scope & Closures

TOPICS COVERED

- Variable scope types: global, function, block
- var vs let vs const — what really changes
- What a closure is and how it forms
- The 3-step closure mechanism
- Private state with the closure pattern
- When and why closures are used

§1 — Variable Scope

The visibility range of a variable

Scope Type	Keyword	Lives Where	Verdict
Global	var (legacy)	Accessible everywhere in the file	Dangerous — pollutes global namespace
Function	var inside function	Only inside that function	Contained but escapes { } blocks
Block	let, const (modern)	Inside { } braces — if, for, while	Safe, predictable, use always

💡 **Golden Rule:**

Always use `let` or `const` — never `var`. `var` escapes if/for/while blocks and creates invisible bugs.

🎯 **KEY TAKEAWAY**

var is the 'rebellious kid' — it escapes { } blocks. let and const stay exactly where you put them. No exceptions, no legacy justification.

§1 — var vs let vs const

Three keywords — three very different behaviours

Behaviour	var	let	const
Reassignable?	✅ Yes	✅ Yes	❌ No
Block scoped?	❌ No (escapes { })	✅ Yes	✅ Yes
Hoisted?	✅ Yes (as undefined)	⚠️ TDZ error	⚠️ TDZ error
Global property?	✅ Yes (window.x)	❌ No	❌ No
Use today?	❌ Never	✅ For mutable values	✅ Default choice

TDZ = Temporal Dead Zone — accessing let/const before its declaration line throws a ReferenceError.

🎯 **KEY TAKEAWAY**

Default to const. Use let only when you need to reassign. Never use var. const prevents accidental reassignment and makes intent clear.

§1 — What Is a Closure?

A function that remembers where it was born

Definition: A closure is an inner function that retains a LIVE REFERENCE to its parent scope's variables — even after the parent function has finished executing.

1

Outer function runs + creates a private variable

2

Outer function returns the inner function

3

Inner function remembers the variable — FOREVER

The parent's variables are NOT destroyed when the parent exits — the closure holds them alive in a hidden 'backpack'.

§1 — Closure in Code

The Coffee Loyalty Card — private counter in action

JavaScript

```javascript
// createLoyaltyCard() is the OUTER function
function createLoyaltyCard() {
  let count = 0;   // PRIVATE — inaccessible from outside

  // These three functions are CLOSURES — they close over 'count'
  return {
    addPurchase : () => ++count,
    getCount    : () => count,
    reset       : () => { count = 0; }
  };
}

const alice = createLoyaltyCard();  // createLoyaltyCard is done — count lives on
const bob   = createLoyaltyCard();  // Independent closure — own private count

alice.addPurchase(); alice.addPurchase();   // alice count = 2
bob.addPurchase();                          // bob count = 1
console.log(alice.getCount());              // → 2  (alice is separate from bob)
console.log(count);                         // ❌ ReferenceError — count is PRIVATE
```

§1 — When Do You Use Closures?

Real-world patterns powered by closures

Private State

Counter or balance hidden from outside code — no one can tamper directly

Factory Functions

Create customised versions: one card per customer, each with its own count

Module Pattern

Expose only a public API — hide internal implementation details

Event Handlers & Timers

Callbacks remember surrounding variables when they fire later

React useState Hook

React closures store component state between renders — literally this pattern

Memoisation

Cache expensive computation results; cache lives in the closure

§1 — Scope & Closures: Summary

- Scope = visibility range of a variable. Block scope (let/const) is the modern standard.
- Closures = inner function retaining access to outer variables after the outer function returns.
- Every closure creates its own independent private scope — alice and bob above are separate.
- React's useState is literally a closure. Master closures and React state clicks immediately.

🎯 **KEY TAKEAWAY**

Closures are the primary mechanism for PRIVATE DATA in JavaScript. They are not an advanced trick — they are the foundation of every UI framework and every module system ever written.

SECTION 2 · 8 MIN

Higher-Order Functions

TOPICS COVERED

- What makes a function 'higher-order'
- .map() — transform every element
- .filter() — keep items that pass a test
- .reduce() — collapse array to one value
- Mental models: transform / sieve / funnel
- Chaining HOFs into data pipelines

§2 — Higher-Order Functions (HOF)

Functions that operate on other functions

Definition: A HOF is a function that ACCEPTS another function as an argument OR RETURNS a function as its result.

.map()

Transform every item

Returns new array
(same length)

.filter()

Keep items passing a test

Returns shorter array
(may be empty)

.reduce()

Collapse to one value

Returns a single value
(number, string, object)

Mental model: map = TRANSFORM filter = SIEVE reduce = FUNNEL

🎯 **KEY TAKEAWAY**

HOFs eliminate repetitive for-loops and make code declarative — you say WHAT to do, not HOW to loop. Every React component uses them.

§2 — .map() — Transform Every Element

Returns a new array of the SAME LENGTH with each element transformed

JavaScript

```javascript
const scores = [88, 43, 76, 55, 91];

// Transform each number into a letter grade
const grades = scores.map(n =>
  n >= 90 ? 'A' : n >= 80 ? 'B' : n >= 70 ? 'C' : n >= 60 ? 'D' : 'F'
);
// → ['B', 'F', 'C', 'F', 'A']   ← same length as input

// Add a new property to every object in an array (spread + map pattern)
const enriched = students.map(s => ({
  ...s,                          // keep all existing properties
  grade: getGrade(s.score),      // add a new 'grade' property
  passed: s.score >= 60          // add a 'passed' boolean
}));

// map never modifies the original array — always returns a brand-new one
```

§2 — .filter() — Keep What Passes

Returns a new array containing ONLY elements where the callback returns true

JavaScript

```javascript
const scores   = [88, 43, 76, 55, 91];
const students = [{name:'Alice',score:88}, {name:'Bruno',score:43},
                  {name:'Cara', score:76}, {name:'Dhruv',score:55}];

// Keep only passing scores (>= 60)
const passing  = scores.filter(n => n >= 60);
// → [88, 76, 91]   (three elements — three pass the test)

// Filter objects — keep students who passed
const passers  = students.filter(s => s.score >= 60);
// → [{name:'Alice',score:88}, {name:'Cara',score:76}]

// Combine: filter THEN map
const passerNames = students
  .filter(s => s.score >= 60)    // step 1: remove failures
  .map(s => s.name);             // step 2: extract names only
// → ['Alice', 'Cara']
```

§2 — .reduce() — Collapse to One Value

Processes every element, accumulating a running result

JavaScript

```javascript
const scores = [88, 43, 76, 55, 91];

// Sum — accumulator starts at 0
const total   = scores.reduce((acc, n) => acc + n, 0);
// Pass:  acc=0 → 88 → 131 → 207 → 262 → 353
// Result: 353

const average = total / scores.length;   // → 70.6

// Build an OBJECT from an array (powerful advanced pattern)
const gradeMap = students.reduce((obj, student) => {
  obj[student.name] = student.score;     // add key:value for each student
  return obj;
}, {});
// → { Alice: 88, Bruno: 43, Cara: 76, Dhruv: 55 }
```

reduce() is the most powerful of the three — it can replicate map() and filter() and build any data structure from an array.

§2 — Chaining HOFs: The Data Pipeline

Compose map, filter, and reduce into readable step-by-step transformations

JavaScript

```javascript
// Find the names of the top 3 passing students, sorted alphabetically
const topStudents = students
  .filter(s  => s.score >= 60)      // Step 1: remove all failing students
  .sort((a,b)=> b.score - a.score)  // Step 2: sort by score descending
  .slice(0, 3)                       // Step 3: keep only top 3
  .map(s    => s.name);             // Step 4: extract just the name string
// → ['Alice', 'Cara', 'Elena']

// Real-world example: dashboard statistics
const stats = {
  total    : students.length,
  passing  : students.filter(s => s.score >= 60).length,
  average  : students.reduce((sum,s)=>sum+s.score,0) / students.length,
  topScore : Math.max(...students.map(s => s.score))
};
```

🎯 **KEY TAKEAWAY**

§2 — Higher-Order Functions: Summary

Method	Input	What It Does	Output
.map(fn)	Array of N items	Runs fn on every element	New array, N items
.filter(fn)	Array of N items	Keeps items where fn returns true	New array, ≤ N items
.reduce(fn, start)	Array of N items	Accumulates fn result across all	Single value
.forEach(fn)	Array of N items	Runs fn for side effects only	undefined (no use)

🎯 **KEY TAKEAWAY**

If you find yourself writing a for-loop to process an array — stop. Ask: can map/filter/reduce solve this? The answer is almost always yes. These three methods are the foundation of every data-driven UI.

SECTION 3 · 7 MIN

The 'this' Keyword & Arrows

TOPICS COVERED

- What 'this' refers to and how it's determined
- The 6 contexts that change 'this'
- The classic callback bug: this = window
- Arrow functions: lexical 'this' explained
- Arrow function syntax: 5 shorthand forms
- What arrow functions do NOT have

§3 — The 'this' Keyword

Its value is determined by HOW the function is called — not WHERE it was written

How Called	this =	Notes
obj.method()	The object obj	Expected, clean
Standalone function()	window (global)	Accidental globals
Regular callback in setTimeout	window — THE BUG ❌	Most common source of bugs
Arrow function anywhere	Lexically inherited from parent	The modern fix ✅
new Constructor()	Newly created object	Class/object creation
.call(obj) / .bind(obj)	obj (forced)	Explicit binding

🎯 **KEY TAKEAWAY**

The rule: 'this' is the object that is calling the function. When there is no object (standalone call, callback), 'this' falls back to window. Arrow functions fix this by having NO own 'this'.

§3 — The Classic 'this' Bug

Regular functions inside callbacks lose the object context

❌ **BROKEN — Regular Function**

```
const cart = {
  items: ['shoes', 'bag'],
  startSale: function() {
    setTimeout(function() {
      // ❌ this = window
      // Not cart!
      this.items.forEach(...)
      // TypeError: Cannot read
      // property of undefined
    }, 1000);
  }
};
```

✅ **FIXED — Arrow Function**

```
const cart = {
  items: ['shoes', 'bag'],
  startSale: function() {
    setTimeout(() => {
      // ✅ this = cart
      // Inherited from
      // startSale's scope
      this.items.forEach(...)
      // Works perfectly!
    }, 1000);
  }
};
```

§3 — Arrow Function Syntax

5 progressively shorter forms — all equivalent in behaviour

Style	Syntax	When to Use
Full traditional	function add(a, b) { return a + b; }	Never in modern code
Arrow multi-line	const add = (a, b) => { return a + b; }	Multiple statements
Arrow single expr	const add = (a, b) => a + b;	Single return value
Single parameter	const double = n => n * 2;	One param — no parens needed
No parameters	const greet = () => 'Hello!';	No args — empty parens required
Return object	const make = id => ({ id, active: true });	Object — wrap in ( ) !

🎯 **KEY TAKEAWAY**

When returning an object literal from a single-expression arrow, you MUST wrap it in ( ) otherwise { } is treated as the function body.

§3 — What Arrow Functions Do NOT Have

4 capabilities that regular functions have — arrows intentionally lack

Missing Feature	Implication	Use Regular Function Instead When…
Own 'this'	Inherits this from surrounding scope	You need the function to define its own context (object methods)
'arguments' object	Cannot access via arguments[0] etc.	You need to iterate all passed arguments (use rest ...args instead)
Can be used with 'new'	Arrow functions are not constructors	You are defining a class or constructor function
Prototype property	No .prototype — no prototype chain	You need to attach methods to a constructor's prototype

- ✅ Use arrow functions for: callbacks, array HOFs, timers, event handlers
- ❌ Avoid arrow functions for: object methods needing own this, constructors

§3 — this & Arrow Functions: Summary

- this = the object calling the function. With no object → this = window.
- Regular functions in callbacks lose context. Arrow functions inherit it.
- Arrow functions = no own this, no arguments, not a constructor, no prototype.
- Default rule: use arrow functions in all callbacks, timers, and event handlers.

🎯 **KEY TAKEAWAY**

When you see a bug where 'this' is undefined or points to window — the fix is almost always: replace function() {} with () => {} in the callback. This single change fixes the most common JavaScript bug category.

SECTION 4 · 9 MIN

Promises & Async/Await

TOPICS COVERED

- The single-thread problem
- Three generations: callbacks → promises → async/await
- Promise states: pending, fulfilled, rejected, settled
- The .then() / .catch() / .finally() chain
- async/await syntax — reading async code like sync
- The 5 rules of async/await

§4 — The Single-Thread Problem

JavaScript has one thread — yet apps must do many things at once

❌ Synchronous — Page FREEZES

```
[Task 1 – runs]
[waiting for server...]
[waiting for server...]
[waiting for server...]
[Task 1 done]
[Task 2 – runs]
[Task 3 – runs]

User clicks nothing – page frozen
```

✅ Asynchronous — Page stays LIVE

```
[Task 1 starts – don't wait]
[Task 2 starts – don't wait]
[Task 3 starts – don't wait]
... other code runs freely ...
[Task 1 done ✓]
[Task 2 done ✓]
[Task 3 done ✓]

User interacts freely – page alive
```

§4 — Three Generations of Async JavaScript

From nested chaos to readable, linear code

PRE-2015

Callbacks

Pass a function to be called later. Produces deeply nested 'callback hell' — unreadable and error-prone.

ES2015
ES6

Promises
.then() / .catch()

A Promise object represents a future value.
Cleaner than callbacks but long chains are still messy.

ES2017
ES8

async / await

Syntactic sugar over Promises. Reads exactly like synchronous code — the modern standard.

✅ **USE THIS**

🎯 **KEY TAKEAWAY**

async/await is Promises in disguise — same mechanism, better syntax. You must know Promises to debug async/await. Learn both.

§4 — Promise States & Lifecycle

A Promise is always in exactly one of four states

State	Meaning	Handler	Can Transition?
PENDING	Started, not done yet	—	Yes
FULFILLED	Completed successfully	.then(result => ...)	No (settled)
REJECTED	Failed with an error	.catch(error => ...)	No (settled)
SETTLED	Fulfilled OR rejected	.finally(() => ...)	N/A

JavaScript

```javascript
// .finally() — ALWAYS runs regardless of outcome
fetch(url)
  .then(r => r.json())           // fulfilled: parse body
  .then(data => renderUI(data))  // fulfilled: use data
  .catch(err => showError(err))  // rejected: handle error
  .finally(() => hideSpinner()); // ALWAYS: cleanup UI
```

§4 — async/await Syntax

Synchronous-looking code that runs asynchronously

JavaScript

```javascript
// 'async' before function = it always returns a Promise
async function loadBooks(query) {
  try {
    // 'await' pauses THIS function — browser stays fully responsive
    const response = await fetch(`https://api.example.com/books?q=${query}`);

    // fetch() does NOT throw on 404/500 — you must check .ok manually
    if (!response.ok) throw new Error(`HTTP ${response.status}`);

    // Parsing the JSON body is also async — await it too
    const data = await response.json();
    return data.items;

  } catch (err) {     // Catches: network errors + our manual throw
    console.error('Fetch failed:', err.message);
    return [];         // Return safe fallback — app never crashes
  } finally {
    hideSpinner();     // ALWAYS runs — whether fetch succeeded or failed
  }
}
```

§4 — The 5 Rules of async/await

Break any of these and your code silently misbehaves

Rule	Explanation	What breaks if violated
async before function	Marks the function as async; it now returns a Promise	await keyword causes a SyntaxError
await inside async only	Cannot use await at top-level (except in modules)	SyntaxError: await is only valid in async functions
await pauses the function	NOT the whole page — only this async function suspends	Misunderstanding causes race conditions
Use try/catch	Without it, rejected Promises cause unhandled rejection warnings	Silent failures; app may crash unpredictably
Always returns a Promise	Even if you return a plain value — it is auto-wrapped	Callers must await the result or chain .then()

🎯 **KEY TAKEAWAY**

async/await is the #1 pattern you will use in every React/Vue project. Data fetching in every modern UI framework is async/await + try/catch + finally.

§4 — Promises & Async/Await: Summary

- JavaScript is single-threaded — async lets it start work and check back later.
- Promises have 3 states: pending → fulfilled (then) or rejected (catch).
- async/await is syntactic sugar over Promises — same mechanics, synchronous reading.
- fetch() does NOT throw on bad HTTP status — always check response.ok manually.
- try/catch/finally is mandatory: catch = handle errors, finally = cleanup always.

🎯 **KEY TAKEAWAY**

The essential trio: fetch() + async/await + try/catch. Write this pattern from memory before Week 07 (APIs). It will appear in every assignment from here forward.

SECTION 5 · 7 MIN

The Event Loop

TOPICS COVERED

- Why the page never freezes during fetch()
- The 4 components: stack, Web APIs, queues, loop
- Step-by-step walkthrough of async execution
- Microtasks (Promises) vs macrotasks (setTimeout)
- Why microtasks run before macrotasks
- Visualising execution order

§5 — The 4 Components of the Event Loop

Together they let a single-threaded language handle async work

Component	What It Is	Handles	Priority
Call Stack	Runs JS code — one frame at a time	Synchronous code, function calls	Immediate — must empty first
Web APIs	Browser's native layer (outside JS)	setTimeout, fetch, DOM events, setInterval	Offloaded — browser handles it
Microtask Queue	High-priority async callbacks	Promise.then(), await, queueMicrotask()	HIGH — drains before macrotask
Callback Queue (Macrotask Queue) (Macrotask Queue)	Normal-priority async callbacks	setTimeout, setInterval, fetch callbacks	Normal — one per loop iteration

🎯 **KEY TAKEAWAY**

The page never freezes during fetch() because fetch runs in Web APIs — completely outside the Call Stack. The stack is free to handle user events.

§5 — Event Loop: Step-by-Step Flow

Tracing exactly how async code moves through the system

1

Code enters Call Stack

Synchronous functions push onto the stack and execute.

2

Async op handed to Web APIs

setTimeout/fetch etc. leave the stack immediately; browser handles them.

3

Stack continues / empties

Remaining synchronous code runs; async work happens in parallel in browser.

4

Async op completes

Timer fires or response arrives — callback is placed in the appropriate queue.

5

Event Loop checks stack

Is the Call Stack empty? Yes → pull next item from Microtask Queue first.

6

Callback runs

Callback pushed to stack, executes, pops off — loop checks again.

§5 — Microtasks vs Macrotasks

Not all async callbacks have equal priority

Queue	Triggered By	Priority	Drains
Microtask Queue	Promise.then(), await, queueMicrotask()	⚡ HIGH	Fully — all microtasks before any macrotask
Macrotask Queue	setTimeout, setInterval, setImmediate, I/O	Normal	One per event loop iteration

JavaScript

```javascript
console.log('1 — synchronous (Call Stack)');

setTimeout(() => console.log('4 — macrotask'), 0);  // Callback Queue
setTimeout(() => console.log('5 — macrotask 2'), 0);

Promise.resolve().then(() => console.log('2 — microtask'));  // Microtask Queue
Promise.resolve().then(() => console.log('3 — microtask 2'));

// OUTPUT: 1, 2, 3, 4, 5
// ALL microtasks drain before ANY macrotask — even with 0ms delay!
```

§5 — The Event Loop: Summary

- JavaScript is single-threaded — the Call Stack handles one thing at a time.
- Async work (fetch, timers) is offloaded to Web APIs — the stack stays free.
- Microtasks (Promises) ALWAYS complete before macrotasks (setTimeout) — even with 0ms.
- The Event Loop's only job: when the stack is empty, push the next queued callback.

Recommended watching: Philip Roberts — 'What the heck is the event loop anyway?'

youtube.com/watch?v=8aGhZQkoFbQ · 26 minutes · The best explanation that exists

🎯 **KEY TAKEAWAY**

Understanding the Event Loop is what separates engineers who write async code from engineers who understand WHY their async code works — or doesn't.

SECTION 6 · 8 MIN

ES6+ Power Features

TOPICS COVERED

- Destructuring objects and arrays
- Destructuring in function parameters
- Spread operator — expand and merge
- Rest operator — collect remainders
- Optional chaining (?.) for safe access
- Nullish coalescing (??) vs the || operator
- Template literals and ES6 shorthand

§6 — ES6+ Version Timeline

ECMAScript 6 (2015) transformed JavaScript from scripting to engineering

Version	Year	Key Features Introduced
ES2015 (ES6)	2015	let/const, arrow functions, classes, modules, destructuring, spread, template literals, Promises
ES2016 (ES7)	2016	Array.includes(), exponentiation operator (**)
ES2017 (ES8)	2017	async/await, Object.entries(), Object.values(), String.padStart()
ES2019 (ES10)	2019	Array.flat(), Array.flatMap(), Object.fromEntries()
ES2020 (ES11)	2020	Optional chaining (?.), Nullish coalescing (??), BigInt, Promise.allSettled()
ES2022 (ES13)	2022	Array.at(), Object.hasOwn(), top-level await in modules
ES2024 (ES15)	2024	Promise.withResolvers(), Object.groupBy(), Array grouping methods

All modern browsers support ES2020+ natively. Babel (babeljs.io) transpiles for older environments.

§6 — Destructuring: Objects

Extract multiple properties in one concise declaration

JavaScript

```javascript
const user = { name: 'Alice', age: 22, city: 'London', role: 'admin' };

// Basic extraction — save properties as same-named variables
const { name, age }      = user;   // name='Alice', age=22

// Rename — save as a different variable name
const { name: fullName } = user;   // fullName='Alice'

// Default value — used only if property is undefined
const { age = 18 }       = user;   // age=22 (property exists, default ignored)
const { score = 0 }      = user;   // score=0 (property missing, default used)

// Nested destructuring
const { address: { city, postcode } } = user;   // city='London'

// Combined: rename + default
const { role: userRole = 'viewer' } = user;     // userRole='admin'
```

§6 — Destructuring: Arrays & Function Parameters

Same power, applied to arrays and function signatures

JavaScript

```javascript
// ── ARRAY DESTRUCTURING ──────────────────────────────────────
const [first, second, ...rest] = [1, 2, 3, 4, 5];
// first=1, second=2, rest=[3,4,5]

// Skip elements with empty commas
const [, , third] = [1, 2, 3];   // third = 3

// Swap two variables without a temp variable
let a = 1, b = 2;
[a, b] = [b, a];                  // a=2, b=1

// ── IN FUNCTION PARAMETERS ───────────────────────────────────
// Instead of: function render(props) { const name = props.name; ... }
function renderCard({ name, score, rank = 1 }) {
  return `<div>${rank}. ${name}: ${score}</div>`;
}
renderCard({ name: 'Alice', score: 88 });  // rank defaults to 1
```

§6 — Spread Operator ...

Expand an array or object into individual elements or properties

JavaScript

```javascript
// ── ARRAYS ──────────────────────────────────────────────
const a = [1, 2, 3];
const b = [...a, 4, 5];                  // [1, 2, 3, 4, 5]  — append
const c = [0, ...a];                     // [0, 1, 2, 3]     — prepend
const d = [...a, ...b];                  // [1,2,3,1,2,3,4,5]— merge
const copy = [...a];                     // [1, 2, 3]         — clone

// ── OBJECTS ─────────────────────────────────────────────
const defaults = { theme: 'dark', lang: 'en', size: 12 };
const user     = { lang: 'fr', name: 'Alice' };

// Merge — later properties OVERRIDE earlier ones
const config = { ...defaults, ...user };
// → { theme:'dark', lang:'fr', size:12, name:'Alice' }
//              ^^^^^ 'fr' overrides 'en'

// Update one property immutably (React pattern):
const updated = { ...state, count: state.count + 1 };  // state unchanged
```

§6 — Rest Operator ...

Collect remaining elements into an array — same syntax, opposite purpose

JavaScript

```javascript
// ── IN FUNCTION PARAMETERS ──────────────────────────────
// Collects ALL arguments into an array called 'numbers'
function sum(...numbers) {
  return numbers.reduce((acc, n) => acc + n, 0);
}
sum(1, 2, 3, 4);   // → 10  (numbers = [1,2,3,4])

// First arg named, rest collected
function log(level, ...messages) {
  messages.forEach(m => console.log(`[${level}] ${m}`));
}
log('ERROR', 'File not found', 'Retrying...');

// ── IN DESTRUCTURING ────────────────────────────────────
const [head, ...tail]         = [1, 2, 3, 4];   // head=1, tail=[2,3,4]
const { name, ...otherProps } = user;            // otherProps has everything else
```

Quick test: function call → Spread (expand) · function definition or destructuring → Rest (collect)

§6 — Optional Chaining ?.

Access deep properties safely — returns undefined instead of crashing

JavaScript

```javascript
// WITHOUT optional chaining — crashes if any parent is null/undefined
const city = user.address.city;         // ❌ TypeError if address is null

// WITH optional chaining — returns undefined safely
const city  = user?.address?.city;      // ✅ undefined (no crash)

// On methods
const upper = str?.toUpperCase();       // ✅ undefined if str is null

// On arrays
const first = arr?.[0];                 // ✅ undefined if arr is null

// Calling optional functions
const result = callback?.();            // ✅ only calls if callback exists

// Combined with nullish coalescing for a fallback
const city = user?.address?.city ?? 'Unknown city';
// → 'Unknown city' only if city is null/undefined; keeps 0 and ''
```

§6 — Nullish Coalescing ?? · Template Literals

Smart fallbacks + modern string interpolation

JavaScript

```javascript
// ── NULLISH COALESCING (??) ──────────────────────────────
// Only returns right side when left is null OR undefined
const val1 = null      ?? 'fallback';  // → 'fallback'
const val2 = undefined ?? 'fallback';  // → 'fallback'
const val3 = 0         ?? 'fallback';  // → 0  (0 is valid!)
const val4 = ''        ?? 'fallback';  // → '' (empty string is valid!)

// 0 || 'fallback' → 'fallback'  (WRONG for numeric inputs)
// 0 ?? 'fallback' → 0           (CORRECT — 0 is a real value)

// ── TEMPLATE LITERALS ────────────────────────────────────
// Old way: 'Hello ' + name + ', score: ' + score + '/100'
const msg  = `Hello ${name}, score: ${score}/100`;

// Multi-line — no \n needed, whitespace preserved as-is
const html = `<div class='card'><h2>${title}</h2></div>`;

// Any expression works inside ${ }
const label = `${items.length} item${items.length !== 1 ? 's' : ''}`;
```

§6 — ES6+ Features: Summary

Feature	Symbol	Core Purpose	Use It When…
Destructuring	{ } / [ ]	Extract multiple values in one line	Working with objects, arrays, or function params
Spread	...	Expand or merge arrays/objects	Cloning, merging, React state updates
Rest	...	Collect remaining items into array	Variadic functions, destructuring leftovers
Optional chaining	?.	Safe deep property access	API data with uncertain structure
Nullish coalescing	??	Fallback for null/undefined only	Numeric/boolean values that may be 0 or false
Template literals	` `	String interpolation + multiline	Any string with dynamic values or HTML

🎯 **KEY TAKEAWAY**

Spread { ...obj } is used in literally every React state update. Optional chaining ?. eliminates 80% of defensive null-check code. Learn these until they are second nature.

SECTION 7 · 6 MIN

Error Handling & Defensive Coding

TOPICS COVERED

- The try / catch / finally pattern
- Built-in JavaScript error types
- Throwing custom errors with new Error()
- Async error handling: await inside try/catch
- Defensive coding: validate before using
- The professional defensive coding checklist

§7 — try / catch / finally

The structured error handling pattern — mandatory in production code

JavaScript

```javascript
try {
  // Code that might fail — network, parsing, user input, type mismatches
  const data = JSON.parse(userInput);  // Throws SyntaxError if invalid
  processData(data);                   // Might throw TypeError or RangeError

} catch (error) {
  // Runs ONLY if the try block throws — error has .name and .message
  console.error(error.name);           // 'SyntaxError', 'TypeError', etc.
  console.error(error.message);        // Human-readable description
  showUserFriendlyMessage(error.message);

} finally {
  // ALWAYS runs — whether try succeeded or catch was triggered
  hideLoadingSpinner();   // These lines execute no matter what
  enableSubmitButton();
  closeNetworkConnection();
}
```

Error Type	When It Occurs

§7 — Throwing Custom Errors

Write code that validates inputs and fails loudly with meaningful messages

JavaScript

```javascript
// Validate inputs BEFORE using them — fail fast with clear messages
function validateAge(age) {
  if (typeof age !== 'number')   throw new TypeError('Age must be a number');
  if (age < 0 || age > 130)      throw new RangeError(`Age ${age} out of 0–130`);
  return true;  // Only reaches here if valid
}

// Usage — caller must handle it
try {
  validateAge('twenty');   // throws TypeError
  validateAge(150);        // throws RangeError
} catch (e) {
  if (e instanceof TypeError)  console.log('Type problem:', e.message);
  if (e instanceof RangeError) console.log('Range problem:', e.message);
}

// Creating a custom Error subclass (advanced — used in large codebases)
class ValidationError extends Error {
  constructor(field, msg) { super(msg); this.name = 'ValidationError'; this.field = field; }
}
```

§7 — Async Error Handling

Errors in async code are caught the same way — just inside async functions

JavaScript

```javascript
// Complete defensive async pattern — use this every time you fetch
async function fetchUser(id) {
  let spinner = null;
  try {
    spinner = showSpinner();

    // 1. Network could fail → catch handles it
    const res = await fetch(`/api/users/${id}`);

    // 2. Bad HTTP status — fetch doesn't throw; we must check manually
    if (!res.ok) throw new Error(`HTTP ${res.status}: ${res.statusText}`);

    // 3. JSON parsing could fail if server returned bad data
    const user = await res.json();

    // 4. Validate the structure before trusting it
    if (!user?.id) throw new Error('Invalid user object received');
    return user;

  } catch (err) {
    logError(err);     // log for debugging
    showToast(err.message); // show user-friendly notification
    return null;       // safe fallback — calling code checks for null
```

§7 — Defensive Coding Checklist

Professional code assumes failure — every time, without exception

- ✅ Validate all function inputs BEFORE using them — never assume correct type
- ✅ Check response.ok after EVERY fetch() call — it does NOT throw on 4xx/5xx
- ✅ Use ?? to provide fallbacks for missing data — do NOT use || on numbers/booleans
- ✅ Never trust external API data — validate structure before accessing deep properties
- ✅ Always use finally for cleanup: hide spinners, enable buttons, close connections
- ✅ Return null or [] as safe fallbacks in catch — app must never crash
- ✅ Throw new Error('message') with context — not just throw 'error'
- ✅ Use instanceof to differentiate error types and give targeted user messages

§7 — Error Handling: Summary

- try = code that might fail; catch = handle the error; finally = cleanup always.
- Error object has .name (type) and .message (description) — use both.
- async/await errors are caught in try/catch just like synchronous errors.
- fetch() does not throw on 404/500 — always check response.ok manually.
- Defensive coding = validate inputs early, fail loudly, return safe fallbacks.

🎯 **KEY TAKEAWAY**

The difference between a hobby app and a production app is not features — it is error handling. Every senior engineer writes error handling first, then the happy path.

Week 05 — Full Summary (1/2)

Concepts §1 through §4 — one line each

Concept	Core Takeaway
Scope	let/const = block scope. var = function scope. Use let/const — always, no exceptions.
Closures	Inner fn retains access to outer variables after outer fn returns → private state.
.map()	Runs callback on every element → returns NEW array of same length. Original unchanged.
.filter()	Keeps elements where callback returns true → new shorter array. Original unchanged.
.reduce()	Accumulates callback across all elements → single value (number, string, object, array).
HOF Chaining	Chain .filter().map().sort().slice() to build readable data transformation pipelines.
this keyword	Value = the calling object. No object? this = window. In arrow fn: inherited from parent.
Arrow functions	No own this — inherits lexically. Use in callbacks/timers. Cannot be constructors.
Promises	Object: pending → fulfilled (.then) or rejected (.catch). .finally always runs.
async/await	Syntactic sugar over Promises. await pauses the fn only. Must wrap in try/catch.

Week 05 — Full Summary (2/2)

Concepts §5 through §7 — one line each

Concept	Core Takeaway
Event Loop	Stack → Web APIs → queue → loop → stack. Microtasks (Promises) run before macrotasks (setTimeout).
Destructuring	Extract multiple values: const { name, age } = user; const [a,b,...rest] = arr;
Spread ...	Expand: [...arr, 4, 5] or { ...obj, key: val }. Used in every React state update.
Rest ...	Collect: fn(...args) or const [head, ...tail]. Same syntax as spread — opposite role.
Optional chaining	user?.address?.city — returns undefined instead of crashing. Chains as many times as needed.
Nullish coalescing	val ?? 'default' — fallback ONLY if null/undefined. Keeps 0, false, '' unlike ||.
Template literals	`Hello ${name}` — interpolation, multi-line, expressions. Replaces string concatenation.
try/catch/finally	try = might fail; catch(err) = handle it; finally = cleanup always runs.
Custom throw	throw new TypeError('msg') — fail loudly with context. Use instanceof to handle types.
Defensive coding	Validate inputs early. Check response.ok. Return null fallbacks. Never crash silently.

Recommended Resources

The definitive learning path for everything covered today

MDN Web Docs

developer.mozilla.org

The definitive JavaScript reference — bookmark this above all others

Modern JS Tutorial

javascript.info

Best free textbook — covers every concept from today in depth

Event Loop Talk

youtu.be/8aGhZQkoFbQ

Philip Roberts: 'What the heck is the event loop anyway?' — 26 min, must watch

TC39 Proposals

tc39.es

Official committee behind ECMAScript — see upcoming features

Can I Use

caniuse.com

Browser support matrix for every JS and CSS feature

ESLint

eslint.org

Catch bugs and enforce style — add to every project

Public APIs

publicapis.dev

Free APIs for practising async/await + fetch

Babel

babeljs.io

Transpile modern ES6+ to older browser-compatible JS

This Week's Homework

Four tasks — each concept only becomes permanent when you build with it

1

CLOSURES

Extend the Coffee Loyalty Card to support multiple customers. Each customer must have their own independent closure with private stamp count.

2

HOFs

Add a 'Top 3 Students' panel to the Grades Dashboard using .sort(), .slice(), and .map() — display names and scores.

3

MDN READING

Read MDN Array.prototype.reduce() — focus on the advanced examples: building objects, grouping, and flattening nested arrays.

4

ASYNC PROJECT

Build a fetch-powered search tool using any free public API from publicapis.dev. Must use async/away, try/catch, and loading state management.

🎯 **KEY TAKEAWAY**

Each concept from today becomes permanent memory the first time you use it in code you wrote yourself.

⚖️ Legal Notice & Disclaimer

- OWNER: Cyber.SoHo — All content, code examples, diagrams, and structure are the intellectual property of Cyber.SoHo.
- COPYRIGHT: © Cyber.SoHo — All Rights Reserved. Permitted for internal educational use only. Reproduction or redistribution without prior written consent is prohibited.
- NO AFFILIATIONS: No financial or organisational relationship with React, Vue, Angular, Google, Moodle, LinkedIn, Spotify, Instagram, or any other mentioned platform. All trademarks remain the property of their respective owners.
- EXTERNAL LINK SAFETY: Verify all external URLs at virustotal.com/gui/home/url before visiting.
- LIABILITY: All code examples are for educational purposes only and are not production-ready. Cyber.SoHo accepts no liability for consequences of use in production environments.